ICMC USP SÃO CARLOS

USP Universidade de São Paulo




# IA, Otimização e IoT para Indústrias e Negócios

## Do Zero ao MLOps

OpAIoT
Optimization • AI • IoT

# Monitoria

Filas, Escalonamento e Escalabilidade:
da teoria à orquestração de tarefas

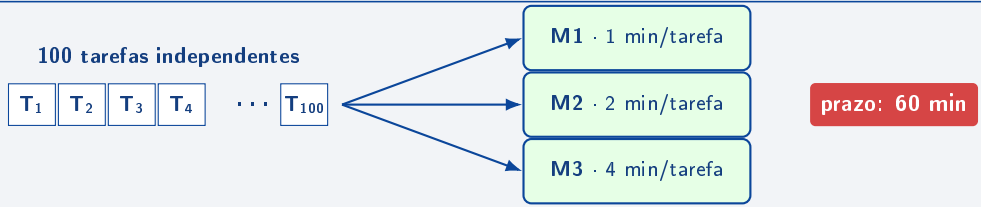
# Roteiro

- A pergunta central
- Filas
- Escalonamento
- Métricas de desempenho
- Escalabilidade e USL
- Prática: filas e escalabilidade
- Orquestração e Kubernetes
- Síntese e discussão

# 1 A pergunta central

OpAIoT
Optimization • AI • IoT

# A pergunta que guia a aula




QUESTÃO

Como distribuir o trabalho para terminar **o mais rápido possível**?
E por que adicionar mais computadores **nem sempre** melhora o desempenho?

**Escalonamento**, **capacidade** e **escalabilidade** são perguntas diferentes.

# Quatro conceitos que não são sinônimos

**Gerenciamento**

Alocar, limitar e monitorar CPU, memória e máquinas entre as aplicações.

**Escalonamento**

Decidir qual tarefa roda em qual recurso e em que ordem.

**Elasticidade**

Adaptar os recursos à demanda, manual ou automaticamente.

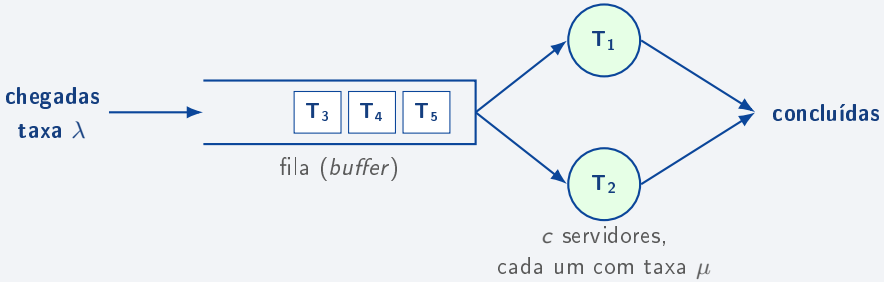
**Escalabilidade**

Quanto a vazão cresce quando se adicionam recursos.

# 2 Filas

OpAIoT
Optimization • AI • IoT

# Anatomia de uma fila




**Atendimento** quem é atendido primeiro: FIFO, prioridade, menor tarefa primeiro...

**Lote ou fluxo** todas no início (makespan) ou chegando a uma taxa $\lambda$ (resposta)

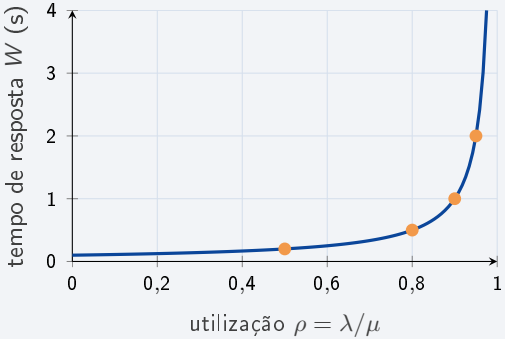
Nosso exemplo é um lote; a fila é a mesma nos dois casos.

# Utilização, lei de Little e saturação

$$\rho = \frac{\lambda}{c\,\mu}$$

$$L = \lambda\, W$$



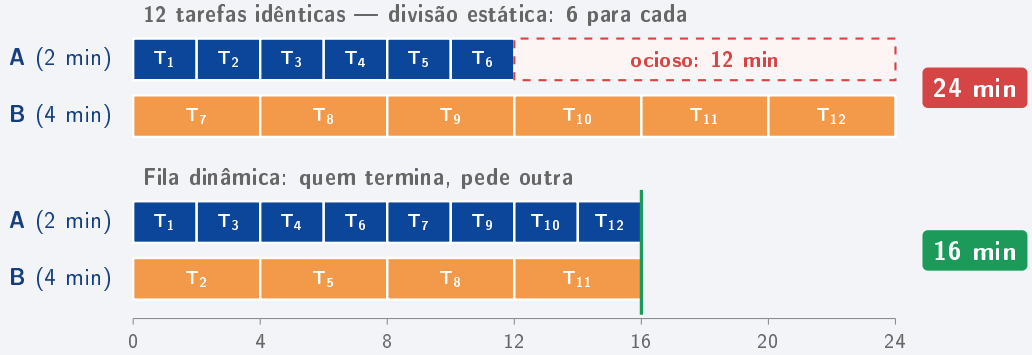

Na M/M/1, $W = 1/(\mu - \lambda)$: de $\rho = 0{,}5$ para 0,9, o **tempo de resposta** (espera + serviço) **quintuplica**.

**Capacidade ociosa não é desperdício: mantém a resposta rápida.**

# 3 Escalonamento

OpAIoT
Optimization • AI • IoT

# Experimento 1 — estática × fila dinâmica




Hipóteses: tarefas disponíveis no instante 0 e sem custo de despacho.

**Balancear o número de tarefas não é balancear a carga.**

# O melhor possível: um limite inferior

$$C_{\max} \geq \frac{N}{\sum_i \mu_i}$$

$\mu_i$: tarefas/min do trabalhador $i$

Experimento 1: $\dfrac{12}{1/2 + 1/4} = 16$ min

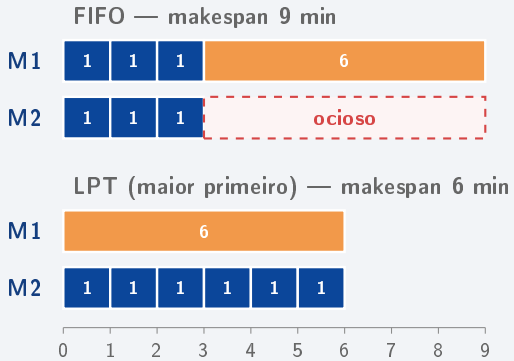
De volta às **100 tarefas** da pergunta central (M1, M2, M3 com 1, 2 e 4 min/tarefa):

| Estratégia | Tarefas (M1, M2, M3) | Makespan | Prazo de 60 min |
|---|---|---|---|
| **Divisão igual** | 34, 33, 33 | 132 min | × |
| **Proporcional à velocidade** | 57, 29, 14 | 58 min | ✓ |
| **Fila dinâmica** | 57, 29, 14* | 58 min* | ✓ |
| **Limite inferior** | supõe trabalho divisível | 57,1 min | |

*Desempate a favor da máquina mais rápida; outro desempate pode dar 56, 29, 15 e 60 min.

# Tamanhos diferentes: a ordem importa

Seis tarefas de 1 min e uma de 6 min, nessa ordem, em duas máquinas iguais:




Minimizar o makespan é **NP-difícil**: usamos heurísticas com garantia (Graham):

$$\text{fila FIFO} \quad \leq \left(2 - \tfrac{1}{m}\right) \cdot \text{ótimo}$$

$$\text{LPT} \quad \leq \left(\tfrac{4}{3} - \tfrac{1}{3m}\right) \cdot \text{ótimo}$$

$m$ máquinas idênticas. Para o tempo **médio** de resposta, o melhor é o oposto: menor primeiro (SPT).

**A métrica escolhida define a melhor política.**

# 4 Métricas de desempenho

# Quatro métricas, aplicadas ao Experimento 1

| Métrica | O que mede | Estática | Dinâmica |
|---|---|---|---|
| **Makespan** $C_{\max}$ | quando termina a última tarefa do lote | 24 min | **16 min** |
| **Vazão** $X$ | tarefas concluídas por unidade de tempo | 0,50/min | **0,75/min** |
| **Utilização** $U$ | fração do tempo ocupado | A 50%, B 100% | **100%, 100%** |
| **Resposta média** $W$ | da chegada à conclusão de cada tarefa | 10,5 min | **9,3 min** |

São métricas distintas: otimizar uma pode piorar outra (LPT × SPT).

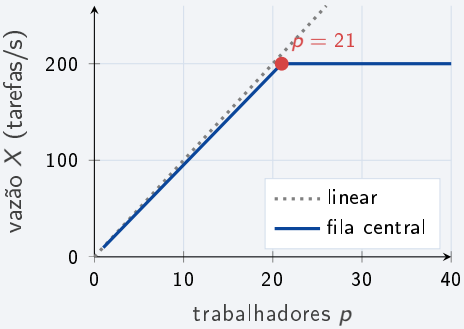
**Mesmo trabalho e mesmos recursos: só a política mudou.**

# O gargalo limita a vazão

Lei da utilização: $U_k = X \cdot D_k$, em que $D_k$ é o tempo que cada tarefa exige do recurso $k$. Como $U_k \leq 1$:

$$X \leq \frac{1}{D_{\max}}$$

**Exemplo:** despacho serial de 5 ms ($X \leq 200$/s); cada trabalhador leva 5 + 100 ms por tarefa. Satura em $p/0{,}105\,\text{s} = 200/\text{s}$: $p = 21$.




**Saturado o gargalo, trabalhadores extras só ficam ociosos.**

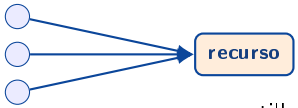
5

# Escalabilidade e a Universal Scalability Law

# USL: contenção e coerência

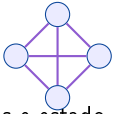
$$C(p) = \frac{X(p)}{X(1)} = \frac{p}{1 + \alpha\,(p-1) + \beta\,p\,(p-1)}$$

**$\alpha$ — contenção**




esperar por um recurso compartilhado:
fila, *lock*, banco, disco, rede

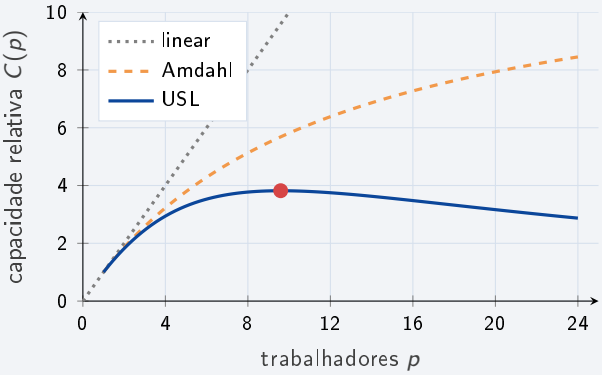
**$\beta$ — coerência**



manter dados e estado consistentes:
custo que cresce com $p(p-1)$

Parâmetros **agregados**, ajustados às medições. $\alpha = \beta = 0$: linear;
$\alpha > 0$, $\beta = 0$ (Amdahl): tende a $1/\alpha$; $\beta > 0$: tem um **máximo** e depois cai.

# Experimento 2 — mais é sempre melhor?




Exemplo teórico ($\alpha = 0{,}08$, $\beta = 0{,}01$), não medições.

$$p^* = \sqrt{\frac{1-\alpha}{\beta}} \approx 9{,}6$$

| $p$ | $C(p)$ | $C(p)/p$ |
|---|---|---|
| 1 | 1,00 | 100% |
| 8 | 3,77 | 47% |
| 10 | 3,82 | 38% |
| 24 | 2,87 | 12% |

**Com 24 trabalhadores, o sistema entrega menos que com 8.**

# De onde vem a USL e como usá-la



- Formulada por **Neil J. Gunther** e fundamentada em modelos de filas, incluindo o ***machine repairman model***: $p$ máquinas que alternam entre trabalhar e esperar por um recurso comum.
- $\alpha$ e $\beta$ são parâmetros **agregados**, ajustados aos dados; $\beta > 0$ não exige que todos os pares troquem mensagens.
- Estimam-se a partir de medições de $C(p)$: por regressão linear após transformar a equação, ou por regressão não linear diretamente em $C(p)$.

$$\frac{p}{C(p)} - 1 \;=\; \alpha\,(p-1) \;+\; \beta\,p\,(p-1)$$

Tarefas independentes não garantem um sistema independente: fila central, disco, banco e memória compartilhados também geram $\alpha$ e $\beta$.

# 6 Prática: uma fila, vários trabalhadores e os limites da escalabilidade

# Roteiro e material da prática

| Atividade | Conceito |
|---|---|
| **Estática × fila dinâmica** | balanceamento de carga |
| **Heterogeneidade e diagrama de Gantt** | makespan, vazão, ociosidade |
| **Aumentar a concorrência** | speedup, contenção, USL |
| **Kubernetes e alternativas** | escalonamento × orquestração |

```
pip install numpy matplotlib jupyterlab
jupyter lab pratica_filas_escalonamento_usl.ipynb                    # simulador, Gantt e USL

python benchmark_filas_cpu.py --tasks 120 --rounds 50000 --workers 1,2,4,6,8 --repeats 3
```

O benchmark grava `medicoes_filas_usl.csv`, lido pelo notebook.

# O que observar

| Experimento | Cenário | Resultado esperado |
|---|---|---|
| **Exp. 1** | 12 tarefas, A e B | 24 → 16 min |
| **Exp. 2** | 100 tarefas, prazo de 60 min | igual: 132 min; fila: 58 min |
| **Exp. 3** | tamanhos log-normais | ÷ limite: estática 2,29; fila 1,05; LPT 1,00 |
| **Despacho** | 5 ms por tarefa de 100 ms | satura em 200 tarefas/s ($p = 21$) |
| **Benchmark** | 8 processos, 120 tarefas iguais | uma execução: $C(8)$ 6,0 (estática), 7,0 (fila) |

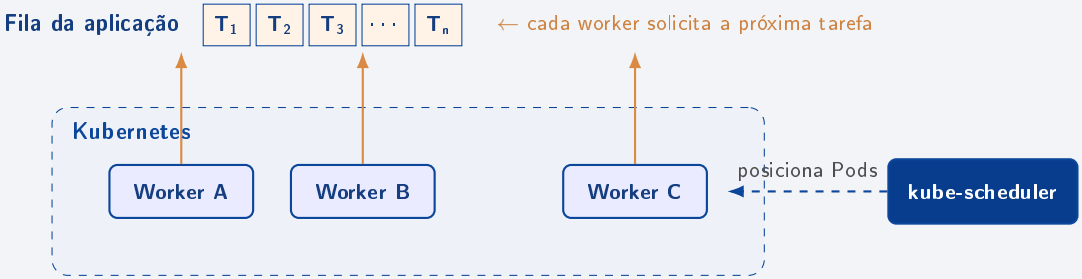
Nessa execução, a fila distribuiu 11, 15, 17, 16, 16, 17, 16 e 12 tarefas. A duração varia (escalonador do SO, interferência, frequência) e a fila compensa. Seus números serão outros.

`--uneven` desequilibra a estática; `--critical-rounds` introduz contenção.

7

# Orquestração: onde entra o Kubernetes

# Escalonamento × orquestração




O `kube-scheduler` decide **em qual Node** cada Pod roda.
**Qual tarefa** cada worker processa é decisão da fila da aplicação.

# Os padrões de Job e o que vimos na aula

| Recurso | Como funciona | Na aula |
|---|---|---|
| ***Indexed Job*** | cada Pod recebe um índice fixo | divisão estática |
| **Job + fila de trabalho** | Pods retiram tarefas de uma fila | fila dinâmica |
| ***Horizontal Pod Autoscaler*** | ajusta as réplicas de um Deployment | muda $p$ na USL |

A documentação traz exemplos dos dois padrões de Job, com RabbitMQ e Redis.
O HPA convencional não escala Jobs; `.spec.parallelism` pode mudar em execução.

**Se a USL tiver um máximo ($\beta > 0$), levar $p$ além de $p^*$ reduz a capacidade.**

# Casos reais: mais recursos sem ganho

| Caso | O que se observou | Causa relatada |
|---|---|---|
| **Ingress-NGINX #5511**<br>AKS, 2020 | 35 → 41 Pods: de 50 mil para 25 mil req/s; p99 de 15 para 40 ms | conexões em TIME_WAIT; mais *keepalive* resolveu (causa não isolada) |
| **go-numa-test**<br>EPYC, 192 vCPUs | máximo com 16 Pods (366 GB/s); com 32 Pods, 175 GB/s (−52%) | memória, *caches* e NUMA |
| **Kubernetes #139111**<br>HPA, 2026 | 50 → 100 workers: de 2.635 para 2.600 reconciliações/s | *mutex* global serializa os workers |
| **Datadog**<br>sistema de jobs | 100 VMs viraram 200 nós no Kubernetes; 138 após ajustes | workers ociosos entre consultas; CPU pedida em excesso |

Os três primeiros mediram vazão estagnada ou em queda; o da Datadog mostra *overhead* de infraestrutura, não perda de vazão. Nenhum ajustou a USL aos dados. Clique no nome para abrir a fonte.

# A pergunta final

QUESTÃO

Se trocarmos nosso gerenciador de workers por Kubernetes, a curva da USL **necessariamente** melhora?

# A pergunta final

QUESTÃO

> Se trocarmos nosso gerenciador de workers por Kubernetes, a curva da USL **necessariamente** melhora?

**Não.**

**Pode melhorar**

- administração dos recursos;
- recuperação de falhas;
- elasticidade.

**Permanece**

- gargalos computacionais;
- contenção ($\alpha$) e coerência ($\beta$);
- o escalonamento das tarefas.

# 8 Síntese e discussão

# Quadro de síntese

| Ideia | Pergunta | Ferramenta | Lição |
|---|---|---|---|
| **Filas** | onde as tarefas esperam? | $\rho$, $L = \lambda W$ | com $\rho \to 1$, a resposta explode |
| **Escalonamento** | quem executa cada tarefa? | fila, LPT, SPT | carga, não contagem |
| **Desempenho** | o que medir? | $W$, $X$, $U$, $C_{\max}$ | a métrica define a política |
| **USL** | até onde escalar? | $\alpha$, $\beta$, $p^*$ | o ganho tem limite |
| **Orquestração** | onde roda cada worker? | kube-scheduler, HPA | Pods $\neq$ tarefas |

**Mais recursos só ajudam até onde a contenção e a coerência permitem.**

# Referências


- N. J. Gunther. *Guerrilla Capacity Planning*. Springer, 2007.
- N. J. Gunther. *A General Theory of Computational Scalability Based on Rational Functions*. arXiv:0808.1431, 2008.
- G. M. Amdahl. *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*. AFIPS Spring Joint Computer Conference, 1967.
- J. D. C. Little. *A Proof for the Queuing Formula: $L = \lambda W$*. Operations Research, 9(3), 1961.
- R. L. Graham. *Bounds on Multiprocessing Timing Anomalies*. SIAM Journal on Applied Mathematics, 17(2), 1969.
- P. J. Denning, J. P. Buzen. *The Operational Analysis of Queueing Network Models*. ACM Computing Surveys, 10(3), 1978.
- M. Harchol-Balter. *Performance Modeling and Design of Computer Systems*. Cambridge University Press, 2013.
- Kubernetes. *Kubernetes Scheduler* e *Jobs* (padrões de fila de trabalho e *Indexed Job*): https://kubernetes.io/docs/




# Obrigado!



opaiot@icmc.usp.br

+55 (16) 99962-7637

MONITORIA

OpAIoT
Optimization • AI • IoT